

Portfolio CMS Platform — Product Requirements Document (PRD)

Product Name

PortfolioOS (*temporary name*)

A dynamic portfolio management platform with a customizable public portfolio website and an authenticated admin dashboard for content, themes, layouts, and section management.

1. Vision

Build a portfolio platform where the user never needs to manually edit code again for content/layout changes.

The system should allow:

- dynamic portfolio customization
- section management
- theme switching
- content editing
- media uploads
- layout personalization
- responsive rendering

through an admin dashboard.

The platform should feel like a simplified mix of:

- [Notion](#)
- [Webflow](#)
- [Framer](#)
- [LinkedIn](#)

but specifically optimized for developer portfolios.

2. Goals

Primary Goals

- Eliminate repeated code edits for portfolio updates
 - Create a reusable portfolio engine
 - Build a scalable architecture
 - Enable dynamic UI rendering
 - Provide complete portfolio customization through admin panel
-

Secondary Goals

- Improve branding flexibility
 - Allow rapid content iteration
 - Create a future SaaS-ready architecture
 - Build a high-quality flagship full-stack project
-

3. Non-Goals (MVP)

The following are NOT included in MVP:

- multi-user support
 - payments/subscriptions
 - AI-generated content
 - collaborative editing
 - website builder drag-drop canvas
 - custom code editor
 - advanced analytics
 - blog CMS
 - public theme marketplace
-

4. User Types

Admin User

Authenticated owner of portfolio.

Capabilities:

- manage sections

- edit content
 - upload media
 - customize themes
 - reorder sections
 - publish changes
-

Public Visitor

Anyone visiting the portfolio website.

Capabilities:

- view portfolio
 - navigate sections
 - view projects
 - contact owner
-

5. Core Product Concept

The platform consists of:

A. Public Portfolio Website

Dynamic portfolio rendered from database configuration.

Example:

abhinav.dev

B. Admin Dashboard

Protected dashboard used to manage:

- portfolio content
- sections
- themes
- layouts
- images
- social links

6. High-Level Architecture

Frontend (React + Vite)

|

| REST API

|

Backend (Node + Express)

|

MongoDB Database

|

Cloudinary (Media Storage)

7. Technical Stack

Frontend

- React
 - Vite
 - Tailwind CSS
 - React Router
 - Zustand (optional later)
-

Backend

- Node.js
 - Express.js
-

Database

- MongoDB
-

Media Storage

- Cloudinary
-

Authentication

- JWT
 - Refresh Tokens
 - Role-Based Access Control
-

8. Functional Requirements

8.1 Authentication

Features

- Login page
 - JWT authentication
 - Protected admin routes
 - Persistent sessions
 - Logout
-

Roles

Admin

Full access

Public

Read-only access

8.2 Portfolio Configuration System

The portfolio must be fully database-driven.

No hardcoded content.

Configuration Includes

- enabled sections
 - section order
 - selected theme
 - typography
 - colors
 - layouts
 - SEO metadata
 - social links
-

8.3 Dynamic Sections System

Admin can:

- enable/disable sections
 - reorder sections
 - edit section content
-

Supported Sections (MVP)

Hero

Fields:

- title
 - subtitle
 - CTA buttons
 - background image
 - profile image
-

About

Fields:

- bio

- highlights
-

Skills

Fields:

- skill name
 - category
 - proficiency
-

Projects

Fields:

- title
 - description
 - technologies
 - github link
 - live link
 - image
-

Education

Fields:

- institution
 - degree
 - dates
 - description
-

Experience

Fields:

- company
 - role
 - duration
 - description
-

Contact

Fields:

- email
 - socials
 - CTA
-

8.4 Theme Engine

Admin Can:

- select predefined themes
 - customize colors
 - customize fonts
 - switch layouts
-

Theme Properties

```
{  
  primaryColor,  
  backgroundColor,  
  textColor,  
  accentColor,  
  fontFamily,  
  borderRadius  
}
```

MVP Themes

- Minimal Dark
 - Glassmorphism
 - Terminal
 - Modern Gradient
-

8.5 Section Ordering

Admin can reorder sections using drag-and-drop.

Example:

Hero
Projects
Skills
Experience
Contact

8.6 Media Management

Features

- upload images
 - crop hero image
 - preview uploads
 - delete media
-

Hero Image Cropper

Requirements:

- aspect ratio lock
- zoom
- crop positioning
- live preview

Suggested library:

- react-easy-crop
-

8.7 Public Portfolio Rendering

The public site must dynamically render:

- enabled sections
- selected layouts

- selected theme

No hardcoded UI assumptions.

8.8 Responsive Design

Must support:

- desktop
 - tablet
 - mobile
-

9. Data Models

9.1 User

```
{  
  email,  
  password,  
  role  
}
```

9.2 Portfolio Config

```
{  
  theme,  
  sectionsOrder,  
  enabledSections,  
  socialLinks,  
  seo  
}
```

9.3 Section

```
{  
  type,  
  enabled,  
  layout,  
  data  
}
```

9.4 Project

```
{  
  title,  
  description,  
  technologies,  
  githubUrl,  
  liveUrl,  
  imageUrl  
}
```

10. API Requirements

Authentication APIs

POST /auth/login
POST /auth/logout
POST /auth/refresh

Portfolio APIs

GET /portfolio
PUT /portfolio

Section APIs

GET /sections
POST /sections
PUT /sections/:id
DELETE /sections/:id

Project APIs

GET /projects
POST /projects
PUT /projects/:id
DELETE /projects/:id

Upload APIs

POST /upload/image

11. UI/UX Requirements

Admin Dashboard

Must feel:

- minimal
- clean
- fast
- modern

Inspired by:

- [Linear](#)
 - [Vercel](#)
 - [Notion](#)
-

Portfolio Frontend

Must support:

- smooth transitions
 - responsive layouts
 - accessibility
 - performance optimization
-

12. Security Requirements

- Password hashing
 - Protected admin APIs
 - Input validation
 - Secure image upload handling
 - Environment variable protection
-

13. Performance Requirements

- Lazy loading
 - Optimized images
 - Code splitting
 - Fast first load
 - Responsive rendering
-

14. MVP Milestones

Phase 1 — Foundation

- React setup
 - Express setup
 - MongoDB connection
 - Authentication system
-

Phase 2 — Admin CMS

- dashboard UI
 - CRUD operations
 - section management
-

Phase 3 — Dynamic Rendering

- render sections dynamically
 - theme engine
 - layout system
-

Phase 4 — Media System

- image uploads
 - hero cropper
 - previews
-

Phase 5 — Polish

- animations
 - responsiveness
 - optimization
 - deployment
-

15. Future Scope

Post-MVP Features

- multi-user support
- custom domains
- portfolio templates

- AI suggestions
 - markdown blogging
 - analytics dashboard
 - resume generator
 - public portfolio sharing
 - SaaS deployment
-

16. Success Criteria

The product is successful if:

- portfolio updates require zero code edits
 - sections can be managed dynamically
 - themes switch instantly
 - public site renders dynamically
 - admin workflow feels intuitive
 - deployment is stable and responsive
-

17. Engineering Principles

MUST FOLLOW

- component reusability
 - schema-driven rendering
 - separation of content and presentation
 - scalable folder structure
 - reusable APIs
 - configuration-first architecture
-

18. Recommended Folder Structure

client/

```
|— src/
|   |— components/
|   |— sections/
|   |— themes/
|   |— layouts/
```

```
| |— pages/  
| |— admin/  
| |— hooks/  
| |— services/  
| |— utils/  
| |— context/
```

server/

```
|— controllers/  
|— middleware/  
|— models/  
|— routes/  
|— config/  
|— utils/
```

19. Final Product Philosophy

This is NOT just a portfolio website.

It is:

- a portfolio operating system
- a customizable rendering engine
- a scalable personal branding platform

The architecture should prioritize:

- flexibility
- maintainability
- extensibility
- long-term scalability